Lookup tables are lists of words that help to extract entities. Here we are defining a list of departments/courses offered in the college, as it will help the bot to reply better based on the student's department.

- **stories.yml**

When the user asks about the course duration, the reply is not just a plain spoken message; instead, it's an action taken.

- **actions.py**

The *actions* file holds the custom action that can run any code you want. This is used to make an API call, or to query a *database*, for example. Rasa framework has a beautifully decoupled actions server; to run it, we need to call 'rasa run actions'. From the example, we are returning the course duration saved from the *dict*, but it could be anywhere.

Output in the UI for this scenario is seen in Figure 4.

Scenario 3

Exam results

nlu.yml

```
- regex: rollnumber
examples: |
- \d{10,30}

- intent: get_roll_number
examples: |
- my roll number is [1234567891](rollnumber)
- This is my roll number [1234567891](rollnumber)
- [1234567891](rollnumber)

- intent: request_result
examples: |
- may I know the exam results
- can you please help me to know if I have passed
- am I all clear
```

rules.yml

```
- rule: activate result form
steps:
- intent: request_result # intent that triggers form
activation
- action: result_form # run the form
- active_loop: result_form # this form is active

- rule: submit form
condition:
- active_loop: result_form # this form must be active
steps:
- action: result_form # run the form
- active_loop: null # the form is no longer active because it
has been filled
- action: utter_submit # action to take after the form is
complete
- action: utter_slots_values # action to take after the form
is complete
- action: action_show_result
```

domain.yml

```
intents:
- get_roll_number
- request_result

actions:
- action_show_result

forms:
result_form:
required_slots:
rollnumber:
- type: from_entity
  entity: rollnumber

slots:
rollnumber:
type: any

entities:
- rollnumber

responses:
utter_result:
- text: "For {roll}, result is {result} with {score} score"

utter_ask_rollnumber:
- text: "Please provide your roll number"

utter_submit:
- text: "All done!"

utter_slots_values:
- text: "I am going to run a result search using the
following parameters:\n
rollnumber: {rollnumber}"
```

actions.py

```
class ActionShowResult(Action):

def name(self) -> Text:
return "action_show_result"

def run(self,
dispatcher: CollectingDispatcher,
```